Informatics II Exam FS 2023

February 15, 2024

1 Single Choice [8 points]

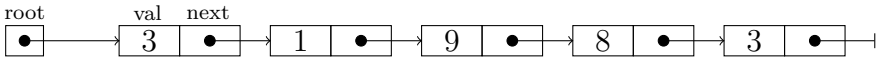
Consider a C program that includes the following declarations and definitions:

```c
struct node {
  int val;
  struct node* next;
};

struct node* root;

void procedureX(struct node* root) {
  if (root->next == NULL) printf("NULL\n");
  else {
    struct node* h = root->next;
    struct node* cur = h->next;
    struct node* t = h;
    while (cur != NULL) {
      struct node* nex = cur->next;
      if (cur->val >= h->val) {
        cur->next = root->next;
        root->next = cur;
      } else {
        t->next = cur;
        t = t->next;
      }
      cur = nex;
    }
    t->next = NULL;
    printf("%d\n", root->next->val);
  }
}
```

Assume root has been instantiated to the following list:



What does the call `procedureX(root)` print?

- ☐ 1
- ☐ 9
- ☐ 8
- ☑ 3
- ☐ NULL

2 Single Choice [4 points]

Assume an array $A[1...n]$ with $n = 9$ elements as follows:

	1	2	3	4	5	6	7	8	9
A =	4	11	5	0	6	1	10	7	8

The operation $exchange(A, i, j)$ that is used in the Heapify algorithm swaps the elements at positions i and j in array A.

```
Algo: Heapify(A,i,s)
1 m = i;
2 l = 2 * i;
3 r = 2 * i + 1;
4 if l≤s ∧ A[l]>A[m] then m = l;
5 if r≤s ∧ A[r]>A[m] then m = r;
6 if i≠m then
7     exchange(A,i,m);
8     Heapify(A,m,s);
```

```
Algo: BuildHeap(A,n)
1 for i = ⌊n/2⌋ to 1 do Heapify(A,i,n);
```

What is the number of exchange operations that the BuildHeap algorithm performs to transform A into a max heap?

- ☐ 2
- ☐ 3
- ☐ 4
- ☑ 5
- ☐ 6

3 Single Choice [8 points]

Consider the red-black tree T in Figure 1:

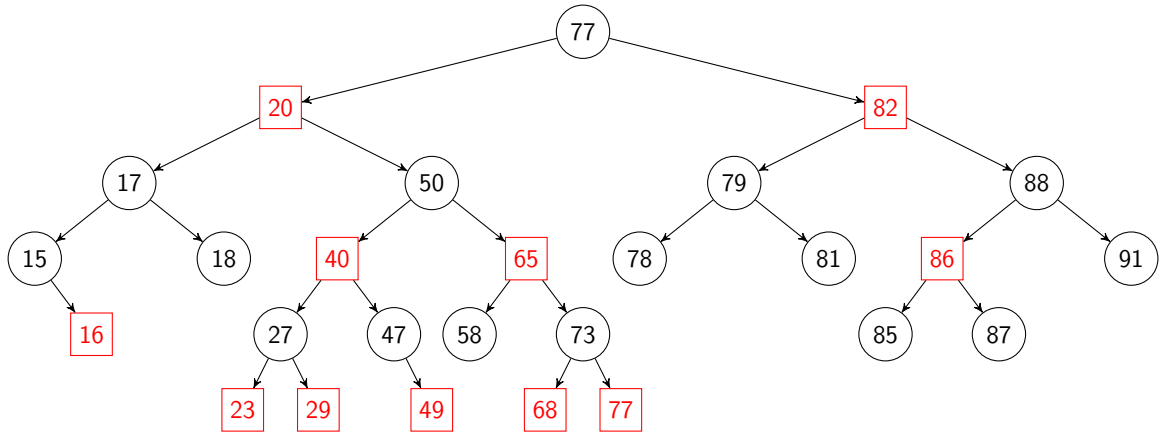


Figure 1: Red-black tree T

Assume a node with key 38 is inserted into T. What is the number of re-coloring operations that must be performed to restore the red-black tree properties? Use the operations discussed in the lecture notes to determine the answer. Only count re-coloring operations that change the color of a node (i.e., if a color assignment does not change the node of a color it is not counted).

- ☐ 3 re-colorings
- ☐ 6 re-colorings
- ☐ 9 re-colorings
- ☑ 10 re-colorings
- ☐ 12 re-colorings

4 Single Choice [6 points]

Consider the recurrence $T(n) = 2T(n/2) + n\log(n) - n + O(\log(n))$ with $T(1) = 1$. Determine the Master method case that applies and the asymptotic complexity it yields.

- ☐ Case 2 applies and yields complexity $\Theta(\log(n))$
- ☐ Case 1 applies and yields complexity $\Theta(n)$
- ☐ Case 3 applies and yields complexity $\Theta(n\log(n))$
- ☐ Case 2 applies and yields complexity $\Theta(n\log(n))$
- ☑ None of the cases of the Master method can be applied.

5 Single Choice [4 points]

Let A[1...n] be an array with n distinct numbers as its elements. A pair (i, j) is called an *inversion* if $i, j \in [1...n]$, $i < j$ and $A[i] < A[j]$. What is the maximum number of inversions that can occur in an array with n elements?

- ☐ n
- ☐ n^2
- ☐ $\frac{n(n+1)}{2}$
- ☑ $\frac{n(n-1)}{2}$
- ☐ $\frac{n^2}{2}$

6 Single Choice [4 points]

How many times does the call `printHello(n)` execute the **printf** statement?

Algo: printHello(n)

```
1 for (i = n; i > 0 ; i = i/3) do
2     for (j = 1; j <= n ; j = j * 2) do
3         printf("Hello World");
```

- ☐ $O(\frac{n}{3} + \frac{n}{2})$
- ☐ $O(\frac{n}{3} \cdot \frac{n}{2})$
- ☐ $O(\frac{n}{3} \cdot \log_2(n))$
- ☑ $O(\log_3(n) \cdot \log_2(n))$
- ☐ $O(\frac{n}{2} \cdot \log_3(n))$

7 Kprim [2 points]

Consider $f(n) = 2^n + n^2$. Determine for each of the following statements whether it is true (check True) or false (check False).

True	False	
☑	☐	$f(n) = O(2^n)$
☑	☐	$f(n) = \Theta(2^{n+1})$
☑	☐	$f(n) = \Omega(n \cdot \log(n))$
☐	☑	$f(n) = O(n^3)$

8 Kprim [12 points]

Assume an array $A[1...n]$ for Boolean values. All values of array A have been initialized to *true*. The algorithm of Erastothenes marks the multiples of integers larger than 1 as non-primes to determine the prime numbers between 2 and n. The following code fragment implements this idea and guarantees that, for all $x \geq 2$, after the execution of the code fragment $A[x]$ is true exactly if x is prime.

Consider a C program that includes the following code fragment:

```
1 for (sX; eX; iX) {
2   for (sY; eY; iY) {
3     A[i*j] = false;
4   }
5 }
```

Determine if, for the values of sX, eX, iX, sY, eY, and iY given below, the above code fragment correctly identifies the prime numbers between 2 and n (check True) or not (check False).

True	**False**	sX	eX	iX	sY	eY	iY
☐	☑	i=2	i<=n/2	i++	j=3	j<=n/i	j++
☑	☐	i=2	i<=n/2	i++	j=2	j<=min(n/i,i)	j++
☑	☐	i=2	i<=sqrt(n)	i++	j=i	j<=n/i	j++
☑	☐	i=n/2	i>=2	i--	j=2	j<=n/i	j++

9 Kprim [10 points]

Consider the red-black tree T in Figure 2:

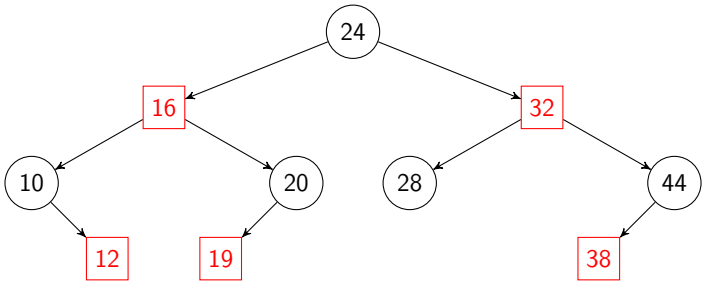


Figure 2: Red-black tree T

Assume the node with key 24 is deleted. What is the number of rotation operations and re-colorings that must be performed to restore the red-black tree properties. If in a step multiple nodes can be selected consider all possibilities. Use the operations described in the lecture notes to determine the answer. Only count re-coloring operations that change the color of a node (i.e., if a color assignment does not change the node of a color it is not counted).

True	**False**	
☐	☑	2 rotations and 3 re-colorings
☑	☐	0 rotations and 1 re-coloring
☐	☑	1 rotation and 3 re-colorings
☑	☐	2 rotations and 5 re-colorings

10 Open Text [10 points]

Assume a directed graph $G = (V, E)$ is traversed in a depth first manner. Write an algorithm that prints the edge type of each edge of the graph during the traversal. The edge types that must be identified are tree edges, back edges, forward edges and cross edges.

Algo: DFStree(u)

```
1  t++;
2  u.stime = t;
3  for v ∈ u.adj do
4      if v.stime == 0 then
5          print "u to v is a Tree edge";
6          DFStree(v);
7      else if v.etime == 0 then print "u to v is a Back edge";
8      else if u.stime < v.stime then print "u to v is a Forward edge";
9      else print "u to v is a Cross edge";
10 t++;
11 u.etime = t;
```

Algo: PrintAllEdgeTypes()

```
1  for v ∈ V do u.stime = 0; u.etime = 0;
2  t = 0;
3  for u ∈ V do
4      if u.stime == 0 then DFStree(u);
```

11 Open Text [12 points]

Consider text string $X[0...n-1]$ with n characters and text string $Y[0...m-1]$ with m characters. The goal is to determine the *edit distance*, i.e., the minimum number of characters that must be inserted into X, deleted from X, and substituted in X by the corresponding character in Y to change string X to string Y.

For instance the edit distance between $X =$ "SGM" and $Y =$ "$AGCM$" is 2 since 'S' in X must be replaced by 'A' in Y, and a 'C' must be inserted in X after 'G'. As another example the edit distance between $X =$ "SGA" and $Y =$ "AG" is 2 since 'S' in X must be replaced by 'A' in Y, and 'A' in X must be deleted.

Give the C function `int ed(char X[], int n, char Y[], int m)` that computes and returns the edit distance between X and Y.

```
int ed(char X[], int n, char Y[], int m) {
  DP[0][0] = 0;
  for (int i = 1; i <= n; i++) { DP[i][0] = DP[i - 1][0] + 1; }
  for (int j = 1; j <= m; j++) { DP[0][j] = DP[0][j - 1] + 1; }
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      if (X[i - 1] == Y[j - 1]) { DP[i][j] = DP[i - 1][j - 1]; }
      else { // delete X[i]
             DP[i][j] = DP[i - 1][j] + 1;
             // insert Y[j]
             DP[i][j] = min(DP[i][j], DP[i][j - 1] + 1);
             // substitute X[i] with Y[i]
             DP[i][j] = min(DP[i][j], DP[i - 1][j - 1] + 1);
           }
    }
  }
  return DP[n][m];
}
```

12 Open Text [10 points]

Provide pseudocode for a function `sortStack` that takes a stack S of integers as parameter, and modifies S by sorting the stack in ascending order such that the smallest integer is at the top.

You are **ONLY** allowed to use the abstract data type `Stack` with the following procedures and functions:

- `initStack()`: initializes and returns a stack
- `push(S,x)`: pushes value x onto stack S
- `pop(S)`: removes and returns the top value from a non-empty stack S
- `isEmpty(S)`: returns True if stack S is empty and False otherwise

```
void StackSort(struct Stack* s) {
  struct Stack *a = initStack(), *b = initStack(), *c;
  while (!isEmpty(s)) { push(a, pop(s)); }
  while (!isEmpty(a)) {
    int maxV = pop(a);
    while (!isEmpty(a)) {
      int tmpV = pop(a);
      if (tmpV > maxV) { push(b,maxV); maxV = tmpV; }
      else { push(b,tmpV); }
    }
    push(s,maxV);
    c = a; a = b; b = c;
  }
  free(a); free(b);
}
```